

Big picture: what a good final-year Python project looks like

A strong final-year project is not just “some code that runs”. It should show that students can:

- Identify a real problem and define it clearly.^{[1][2]}
- Design a solution (requirements, architecture, data model, UI, etc.) before coding.^[1]
- Implement that solution in clean, well-structured Python code that others can read, run, and extend.^{[3][4]}
- Test it properly and show evidence that it works reliably.^{[5][1]}
- Document everything: report, code documentation, and a short user manual or demo.^{[2][6][5]}

Below is a step-by-step mentoring guide you can give your batch, with simple examples at each stage.

Step 1: Choosing and defining the problem

Students often jump straight into coding without being clear on the problem. Stop them here and make them write it down.

What they should do

1. Pick a realistic problem:
 - Something that can be done in 3–4 months with your team size.
 - Has a clear user (students, teachers, shop owners, hostel warden, etc.).
2. Write a **problem statement**:

- What is the problem?
- Who faces it?
- What is currently done (existing system)?
- Why is that not enough?^[2]

Mini example

Suppose they choose: “Hostel Mess Feedback System”.

A simple problem statement:

- Students have complaints about food quality and mess timings, but feedback is informal (WhatsApp, verbal), so issues are lost.
- There is no structured way for wardens to see trends (e.g., which meal gets most complaints).

Ask them to write 1–2 paragraphs and a bullet list of pain points. This becomes the first section of their report (Introduction + Problem Statement).^[2]

Your mentoring question to students

“Write your problem statement in 5–8 lines: what is the problem, who has it, and what is wrong with current solutions?”

Step 2: Requirements and basic design

Once the problem is clear, move to **requirements** and a **rough design** before coding.

2.1 Functional and non-functional requirements

Ask them to write:

- Functional requirements (what the system must do):
 - Students can submit feedback with rating and comments.
 - Warden can view feedback by date and meal.
 - Admin can export feedback as CSV.
- Non-functional requirements (how it should behave):
 - System should respond within 2 seconds for typical queries.
 - Only authorized admin can view all feedback.
 - App should be simple enough for non-technical users.^{[1][2]}

2.2 High-level design

They should draw simple diagrams:

- Architecture: e.g., “Frontend (web or CLI) → Flask API → SQLite database”.
- Data model: tables/classes and their fields.
- Major modules: auth, feedback, reports, etc..^[1]

You can tell them: “Your design section in the report should have these diagrams and explanations”.^[2]

Your mentoring question

“List at least 5 functional and 3 non-functional requirements for your idea. Which one do you think will be hardest to implement?”

Step 3: Setting up a clean Python project

Teach them to set up a **professional** project, not just one big .py file.

3.1 Project structure

Show a simple, standard layout for a small app:

```
mess_feedback/  
  README.md  
  requirements.txt  
  src/  
    __init__.py  
    app.py  
    feedback.py  
    database.py  
  tests/  
    test_feedback.py
```

- src/ holds application code.
- tests/ holds unit tests.
- requirements.txt lists external libraries.

Such structure aligns with modern Python practices and makes collaboration easier.^{[4][3]}

3.2 Virtual environment and dependencies

Ask them to:

- Create a virtual environment (e.g., `python -m venv venv`) and activate it.
- Install dependencies like Flask, SQLAlchemy, etc., and add them to requirements.txt.

Using a project tool or environment manager helps keep dependencies consistent.^[4]

Micro coding example (very simple)

src/app.py:

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Mess Feedback System - Home Page"

if __name__ == "__main__":
    app.run(debug=True)
```

This is just to show how to start something cleanly in a file inside src/.

Your mentoring question

“Create a folder structure for your project and list the names of the main Python files you plan to have. What will each file roughly contain?”

Step 4: Writing clean, readable Python code

You can insist on code quality as part of the grade. Explain that good code is:

- Readable: clear names, consistent style.

- Modular: logic broken into small functions/classes, not huge scripts.
- Maintainable: easy to change or extend.^[5]

4.1 Naming and modularity

Example: feedback logic in feedback.py:

```
from datetime import datetime

def validate_feedback(rating: int, comment: str) -> bool:
    """Return True if rating and comment are valid, else False."""
    if rating < 1 or rating > 5:
        return False
    if len(comment.strip()) == 0:
        return False
    return True

def create_feedback_entry(user_id: int, rating: int, comment: str) -> dict:
    """Create a feedback dictionary with timestamp."""
    if not validate_feedback(rating, comment):
        raise ValueError("Invalid feedback data")

    return {
        "user_id": user_id,
        "rating": rating,
        "comment": comment.strip(),
        "timestamp": datetime.now().isoformat()
    }
```

Here students learn:

- Functions do *one* clear thing.
- Useful docstrings explaining purpose.^[6]
- Type hints `int`, `str`, `dict` to improve clarity.^{[6][5]}

4.2 Style tools

Encourage:

- Using a formatter (e.g., `black`) and linter (e.g., `ruff`, `flake8`, `pylint`) so style and basic errors are caught automatically.^[5]
- Following PEP 8 (Python style guide) and PEP 257 (docstrings).^{[6][5]}

Your mentoring question

“Take one function from your project idea and write it with a clear name, type hints, and a docstring. What does it do in one sentence?”

Step 5: Testing and reliability

Engineering projects must show testing, not just “we manually tried it”.

5.1 Unit tests with `pytest` or `unittest`

Ask them to write unit tests for core logic, like `validate_feedback`.

Example using `pytest` in `tests/test_feedback.py`:

```
from src.feedback import validate_feedback

def test_validate_feedback_accepts_valid_data():
    assert validate_feedback(4, "Good food today") is True
```

```
def test_validate_feedback_rejects_out_of_range_rating():
    assert validate_feedback(6, "Too salty") is False

def test_validate_feedback_rejects_empty_comment():
    assert validate_feedback(3, " ") is False
```

This shows:

- How to verify behavior automatically.
- How to separate test code from main code.^[5]

5.2 Other testing

Encourage at least:

- Functional testing: “Follow these steps and see if the feature works end-to-end.”
- Recording test cases in the report (test case ID, input, expected output, actual output).^{[2][5]}

Your mentoring question

“What are three critical functions or features in your project that you must write tests for, and why are they critical?”

Step 6: Documentation and report

Documenting is as important as coding for a final-year project.

6.1 Project documentation (in repo)

At minimum:

- README.md: overview, features list, how to install and run, tech stack used.
- Docstrings in code for key modules and functions.^{[6][5]}
- Basic user instructions: how to log in, how to use main features.

6.2 Formal report structure

Most colleges expect a structured report including: cover page, introduction, problem statement, existing system analysis, requirements, design, testing, implementation, user manual, source code appendix, and references.^[2]

Guide students to keep each section short but clear, with diagrams (system architecture, ER diagram, flowcharts) in the design chapter.^[2]

Your mentoring question

“Outline the main chapters of your report and write 1–2 lines on what you will put in each chapter.”

Step 7: Teamwork, planning, and version control

Many capstone handbooks emphasize that students must follow an engineering process: requirements, design, implementation, testing, deployment, and regular progress reviews.^{[7][1]}

7.1 Version control (Git)

Teach basic Git workflow:

- Create a GitHub/GitLab repository.
- Each student works on a branch and creates merge requests.
- Use commit messages that describe changes clearly.

7.2 Planning and milestones

Ask teams to:

- Break work into milestones:
 - Week 1–2: requirements + design
 - Week 3–5: basic prototype
 - Week 6–8: features completion
 - Week 9–10: testing, polishing, documentation
- Meet weekly, track tasks (e.g., Trello, Jira, or a simple shared Google Sheet).^{[8][9]}

This aligns with expectations that capstone projects involve serious engineering, not ad-hoc coding.^{[8][11]}

Your mentoring question

“How will you divide responsibilities in your team (e.g., backend, frontend, testing, documentation), and what will each member deliver by the first month?”

Step 8: Presentation and demo

Finally, they must be able to *demonstrate* their work clearly.

Ask them to prepare:

- A 10–15 minute presentation: problem, solution, architecture, main features, challenges, and learning outcomes.^{[10][8]}
- A live demo: pre-planned scenario that shows the main flows without depending on risky last-second changes.
- Screenshots for the report and backup in case network or server has problems.^{[10][2]}

Encourage them to practice explaining “Why did we design it this way?” and “What could be improved in future work?”.

What students should pay special attention to

Summarize these as grading dimensions:

- **Clarity of problem and requirements** (not vague).
- **Sound design** (architecture, data model, clear modules).
- **Code quality** (structure, naming, modularity, documentation, testing).^{[3][4][5]}
- **Engineering process** (planning, version control, teamwork, iterations).^{[7][8][1]}
- **Documentation and presentation quality** (report, README, user manual, demo).^{[10][2]}

These are exactly the aspects many capstone guidelines and code-quality guides highlight for serious software projects.^{[7][1][5]}

To make this truly useful for your batch: what kind of projects are your students leaning towards right now (web apps, data science/ML, automation tools, something

with hardware), and at what level are they in Python so far (basic scripts, OOP, web frameworks, ML libraries)?

**

1. <https://www.eecs.uottawa.ca/~tcl/seg4910-11/rules-e/>
2. <https://www.scribd.com/document/309848770/Capstone-Project-Report-Format>
3. <https://dagster.io/blog/python-project-best-practices>
4. <https://www.stuartellis.name/articles/python-modern-practices/>
5. <https://realpython.com/python-code-quality/>
6. https://www.reddit.com/r/Python/comments/11e00kq/code_quality_best_practices/
7. https://ece.uwaterloo.ca/~se_capstone/se-capstone-handbook.pdf
8. <https://github.com/Gemarodri/Capstone499>
9. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8063748/>
10. <https://www.scribd.com/document/983028590/MIS-451-Capstone-Project-Guidelines-Sem-2>